



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

PORTAFOLIO DE EVIDENCIAS

ASIGNATURA:
“Ciencia de Datos”

PRESENTA:

Dimas Montoro Héctor Iván 22350317

CARRERA:
**INGENIERÍA EN SISTEMAS
COMPUTACIONALES**

DOCENTE:
Julio Agilar Carmona

Mayo 2026

Día 1.....	2
Podcast.....	2
Configuramos el entorno.....	4
Creamos el agente.....	5
Ejecutamos el agente.....	6
Prueba.....	7
Probando interfaz web de ADK.....	8
Día 2.....	9
Parte 1.....	9
Podcast.....	9
Custom tools.....	11
¿Cómo definir las?.....	12
Sección 3.....	15
Diferencia entre Agent Tools y Sub-agents.....	17
Parte 2.....	18
Model Context Protocol.....	18
¿Cómo funciona?.....	18
Usando MCP junto al agente.....	20
¿Cómo funciona?.....	24
Crear el agente, la app y el runner.....	26
Construyendo el Workflow.....	27
Día 3.....	29
Gestión de sesiones.....	29
Sesiones persistentes con DatabaseSessionService.....	32
Compactación de contexto.....	35
Día 4.....	37
Día 5.....	39

Día 1

Podcast

Este video ofrece una visión general sobre la arquitectura y el funcionamiento de los agentes de IA, basándose en un whitepaper del curso de Google X Kaggle. El cambio fundamental es pasar de modelos pasivos a sistemas autónomos y orientados a objetivos capaces de planificar y actuar.

Componentes clave de un agente:

- **Modelo:** El LLM que razona y gestiona el contexto.
- **Herramientas:** Conexiones externas como APIs o bases de datos para realizar tareas.
- **Capa de orquestación:** El componente que gestiona el ciclo de planificación, ejecución y observación .

Taxonomía de los agentes

Nivel 0: Modelo base sin herramientas.

Nivel 1 (Conectado): Capaz de usar herramientas para obtener información en tiempo real.

Nivel 2 (Estratégico): Utiliza context engineering para resolver tareas complejas de varios pasos.

Nivel 3 (Colaborativo): Sistema multi-agente donde los agentes delegan tareas a otros especialistas.

Nivel 4 (Auto-evolutivo): La frontera actual, donde el sistema identifica sus carencias y crea nuevas herramientas o agentes para suplirlas.

Aspectos críticos para producción:

Agent Ops: La importancia de evaluar la calidad usando otros modelos (LLM-as-a-judge) y utilizar trazas de telemetría para depurar.

Seguridad: Implementación de defensa en profundidad mediante reglas de código (guardrails) y modelos de protección para evitar comportamientos no deseados.

Gobernanza: Uso de puertas de enlace (gateways) para gestionar el agente sprawl y centralizar la autenticación y permisos.

El video concluye que construir agentes exitosos requiere rigor de ingeniería, arquitectura robusta y una transición del rol del desarrollador hacia el de un arquitecto o director de sistemas autónomos.

Configuramos el entorno

Autenticamos

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets. Details: {e}"
    )
```

✅ Gemini API key setup complete.

+ Code + Markdown

Importamos los componentes

```
from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

Definimos las funciones de ayuda

```
display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")
```

✅ Helper functions defined.

Creamos el agente

Definimos el agente

```
root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")
```

✅ Root Agent defined.

+ Code + Markdown

Ejecutamos el agente

Creamos un runner

```
runner = InMemoryRunner(agent=root_agent)

print("✅ Runner created.")
```

✅ Runner created.

+ Code + Markdown

Usamos el método .run_debug() para poder interactuar con el agente.

```
31: response = await runner.run_debug(
    "What is Agent Development Kit from Google? What languages is the SDK available in?"
)
```

Created new session: debug_session_id

User > What is Agent Development Kit from Google? What languages is the SDK available in?

helpful_assistant > The Google Agent Development Kit (ADK) is an open-source framework designed to simplify the end-to-end development, debugging, and deployment of AI agents and multi-agent systems. It applies software development principles to AI agent creation, allowing developers to build everything from personal AI assistants to complex business workflows. The ADK is optimized for Google's Gemini models and the Vertex AI platform but is model-agnostic and deployment-agnostic, meaning it can be used with other frameworks and deployed in various environments.

The ADK is available in the following programming languages:

- * Python
- * TypeScript
- * Go
- * Java

Prueba

A continuación probaremos el agente con 3 preguntas: ¿Cómo es el clima en Londres?, ¿Quién ganó la última copa mundial de soccer?, ¿Cuales películas nuevas hay en cines?

```
response = await runner.run_debug("What's the weather in London?")
```

Continue session: debug_session_id

User > What's the weather in London?

helpful_assistant > The weather in London is currently clear with a temperature of 52°F (11°C). The humidity is around 66%, and it feels like 49°F (10°C).

For the upcoming days:

* **Thursday, April 30th:** Sunny during the day and clear at night, with temperatures ranging from 51°F (11°C) to 70°F (21°C). Some sources indicate a high of 68°F (20°C) and a low of 52°F (11°C).

* **Friday, May 1st:** Sunny during the day and cloudy at night, with a 15% chance of rain during the day and a 30% chance at night. Temperatures will be between 50°F (10°C) and 74°F (23°C). Another forecast shows sunny weather with a high of 78°F (26°C).

* **Saturday, May 2nd:** Cloudy during the day and light rain at night, with a 25% chance of rain during the day and 40% at night. Temperatures will range from 52°F (11°C) to 65°F (18°C).

Please note that different sources may provide slightly varying forecasts.

+ Code + Markdown

```
response = await runner.run_debug("Who won the last soccer world cup?")
```

Continue session: debug_session_id

User > Who won the last soccer world cup?

helpful_assistant > Argentina won the last soccer World Cup in 2022, defeating France in the final match. This was Argentina's third World Cup title. The final is widely considered one of the greatest in the tournament's history, ending in a 3-3 draw after extra time and decided by a penalty shootout, which Argentina won 4-2.

+ Code + Markdown

```
response = await runner.run_debug("What new movies are showing in theaters now?")
```

Continue session: debug_session_id

User > What new movies are showing in theaters now?

helpful_assistant > There are several new movies showing in theaters now. Some of the recently released films include:

- * *Michael*
- * *The Super Mario Galaxy Movie*
- * *Project Hail Mary*
- * *Lee Cronin's The Mummy*
- * *The Drama*
- * *You, Me & Tuscany*
- * *Hoppers*
- * *Over Your Dead Body*
- * *Mother Mary*
- * *Animal Farm*
- * *Deep Water*
- * *The Devil Wears Prada 2*
- * *Hokum*
- * *That Time I Got Reincarnated as a Slime the Movie: Tears of the Azure Sea*
- * *RZA's One Spoon of Chocolate*

Upcoming releases include *Mortal Kombat II*, *The Sheep Detectives*, and *Star Wars: The Mandalorian & Grogu*.

For specific showtimes and to purchase tickets, you can check websites like Regal, Cinemark, Fandango, or Showcase Cinemas.

+ Code + Markdown

Probando interfaz web de ADK

Generamos un folder que contenga todos los archivos necesarios

```
!adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

Non-empty folder already exist: '/kaggle/working/sample-agent'
Override existing content? [y/N]: ^C
Aborted!

+ Code + Markdown

Creamos una URL personalizada

```
url_prefix = get_adk_proxy_url()
```

⚠ IMPORTANT: Action Required

The ADK web UI is **not running yet**. You must start it in the next cell.

1. **Run the next cell** (the one with `!adk web ...`) to start the ADK web UI.
2. Wait for that cell to show it is "Running" (it will not "complete").
3. Once it's running, **return to this button** and click it to open the UI.

(If you click the button before running the next cell, you will get a 500 error.)

[Open ADK Web UI \(after running cell below\) ↗](#)

Ejecutamos ADK web

```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [101]
INFO: Waiting for application startup.
```

```
+-----+
| ADK Web Server started
|
| For local testing, access at http://127.0.0.1:8000.
+-----+
```

```
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
^C
INFO: Shutting down
INFO: Waiting for application shutdown.
```

```
+-----+
| ADK Web Server shutting down...
+-----+
```

```
INFO: Application shutdown complete.
INFO: Finished server process [101]
```

Día 2

Parte 1

Podcast

explora cómo dotar a los modelos de lenguaje (LLM) de la capacidad de interactuar con el mundo real mediante el uso de herramientas y el Protocolo de Contexto de Modelo (MCP).

Puntos clave discutidos:

- **Naturaleza de las herramientas:** Las herramientas permiten que los modelos pasen de solo "pensar" a "hacer". Se clasifican en tres tipos: funciones definidas por el desarrollador, herramientas integradas y agentes como herramientas.
- **Buenas prácticas de diseño:** Se enfatiza la importancia de una documentación clara (nombres y descripciones descriptivas), enfocarse en tareas en lugar de llamadas a API complejas, diseñar salidas concisas para evitar saturar el contexto del modelo y manejar errores de forma instructiva.
- **Model Context Protocol (MCP):** Este estándar abierto busca solucionar el problema de interoperabilidad al conectar modelos con herramientas. Se basa en una arquitectura cliente-servidor (host, cliente y servidor) usando JSON-RPC 2.0 para estandarizar la comunicación.

Desafíos de escalabilidad y seguridad:

- **Escalabilidad:** Para evitar el "bloat" (exceso) en el contexto del modelo al tener cientos de herramientas, se propone el uso de búsqueda semántica (tipo RAG) para cargar solo las herramientas necesarias.
- **Seguridad:** Se advierte sobre el riesgo del "confuso diputado" (confused deputy), donde un usuario malintencionado podría engañar al modelo para ejecutar acciones no autorizadas. La recomendación es rodear el protocolo

con capas de seguridad empresarial, como puertas de enlace (API gateways), para gestionar la autenticación y autorización.

Custom tools

¿Qué son?

Los **Custom Tools** son herramientas que tú mismo desarrollas utilizando tu propio código y lógica de negocio. A diferencia de las herramientas integradas que ya vienen listas dentro de ADK, estas te permiten tener control total sobre lo que el agente puede hacer y cómo lo hace.

Mientras que las herramientas predefinidas, como las de búsqueda, están diseñadas para cubrir necesidades generales, los custom tools se crean para resolver problemas específicos. Esto es especialmente útil cuando trabajas con requerimientos particulares que no pueden ser cubiertos por soluciones genéricas, como reglas internas de una empresa, acceso a bases de datos privadas o integración con sistemas propios.

Se utilizan principalmente cuando necesitas un alto nivel de personalización. Permiten adaptar el comportamiento del agente a un contexto concreto, automatizar procesos internos o implementar lógica especializada que refleje las necesidades reales del negocio.

Una de sus principales ventajas es la flexibilidad, ya que puedes diseñarlos exactamente según lo que necesitas. Además, facilitan la integración directa con infraestructura existente y mejoran la precisión en tareas específicas al incorporar conocimiento del dominio.

Dentro de ADK, los custom tools funcionan como extensiones del agente que le permiten interactuar con el mundo real, ejecutar acciones y resolver tareas más allá de solo generar texto. En esencia, convierten al agente en un sistema más completo, capaz de operar dentro de entornos reales y no solo responder consultas.

¿Cómo definir las?

Para definir una tool en ADK no necesitas nada complejo: cualquier función de Python puede convertirse en una herramienta del agente siempre que siga ciertas reglas. Básicamente, primero creas una función en Python, luego te aseguras de aplicar buenas prácticas, y finalmente la agregas a la lista `tools=[]` del agente. A partir de ahí, ADK se encarga automáticamente de integrar y hacerla usable por el modelo.

Lo importante no es solo que la función exista, sino cómo está estructurada. Una buena práctica clave es que la función devuelva siempre un diccionario con un formato consistente. Por ejemplo, en caso de éxito algo como `{"status": "success", "data": ...}`, y en caso de error `{"status": "error", "error_message": ...}`. Esto permite que el agente entienda claramente qué ocurrió y cómo continuar.

Otro elemento fundamental son los docstrings. No son solo documentación para humanos: el modelo los utiliza directamente para decidir cuándo usar la herramienta y cómo hacerlo. Por eso deben ser claros, descriptivos y específicos.

```
[15]: # Pay attention to the docstring, type hints, and return value.
def get_fee_for_payment_method(method: str) -> dict:
    """Looks up the transaction fee percentage for a given payment method.

    This tool simulates looking up a company's internal fee structure based on
    the name of the payment method provided by the user.

    Args:
        method: The name of the payment method. It should be descriptive,
            e.g., "platinum credit card" or "bank transfer".

    Returns:
        Dictionary with status and fee information.
        Success: {"status": "success", "fee_percentage": 0.02}
        Error: {"status": "error", "error_message": "Payment method not found"}
    """
    # This simulates looking up a company's internal fee structure.
    fee_database = {
        "platinum credit card": 0.02, # 2%
        "gold debit card": 0.035, # 3.5%
        "bank transfer": 0.01, # 1%
    }

    fee = fee_database.get(method.lower())
    if fee is not None:
        return {"status": "success", "fee_percentage": fee}
    else:
        return {
            "status": "error",
            "error_message": f"Payment method '{method}' not found",
        }

print("✅ Fee lookup function created")
print(f"🧪 Test: {get_fee_for_payment_method('platinum credit card')}")

✅ Fee lookup function created
🧪 Test: {'status': 'success', 'fee_percentage': 0.02}
```

Creamos una segunda herramienta

```
def get_exchange_rate(base_currency: str, target_currency: str) -> dict:
    """Looks up and returns the exchange rate between two currencies.

    Args:
        base_currency: The ISO 4217 currency code of the currency you
            are converting from (e.g., "USD").
        target_currency: The ISO 4217 currency code of the currency you
            are converting to (e.g., "EUR").

    Returns:
        Dictionary with status and rate information.
        Success: {"status": "success", "rate": 0.93}
        Error: {"status": "error", "error_message": "Unsupported currency pair"}
    """

    # Static data simulating a live exchange rate API
    # In production, this would call something like: requests.get("api.exchangerates.com")
    rate_database = {
        "usd": {
            "eur": 0.93, # Euro
            "jpy": 157.50, # Japanese Yen
            "inr": 83.58, # Indian Rupee
        }
    }

    # Input validation and processing
    base = base_currency.lower()
    target = target_currency.lower()

    # Return structured result with status
    rate = rate_database.get(base, {}).get(target)
    if rate is not None:
        return {"status": "success", "rate": rate}
    else:
        return {
            "status": "error",
            "error_message": f"Unsupported currency pair: {base_currency}/{target_currency}",
        }

print("✅ Exchange rate function created")
print(f"🧪 Test: {get_exchange_rate('USD', 'EUR')}")
```

✅ Exchange rate function created
🧪 Test: {'status': 'success', 'rate': 0.93}

Activar Windows
Ve a Configuración para activar Windows.

```
# Currency agent with custom function tools
currency_agent = LLMAgent(
    name="currency_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""You are a smart currency conversion assistant.

    For currency conversion requests:
    1. Use 'get_fee_for_payment_method()' to find transaction fees
    2. Use 'get_exchange_rate()' to get currency conversion rates
    3. Check the "status" field in each tool's response for errors
    4. Calculate the final amount after fees based on the output from 'get_fee_for_payment_method' and 'get_exchange_rate' methods and provide a clear breakdown.
    5. First, state the final converted amount.
       Then, explain how you got that result by showing the intermediate amounts. Your explanation must include: the fee percentage and its
       value in the original currency, the amount remaining after the fee, and the exchange rate used for the final conversion.

    If any tool returns status "error", explain the issue to the user clearly.
    """,
    tools=[get_fee_for_payment_method, get_exchange_rate],
)

print("✅ Currency agent created with custom function tools")
print("🔧 Available tools:")
print("  • get_fee_for_payment_method - Looks up company fee structure")
print("  • get_exchange_rate - Gets current exchange rates")
```

✅ Currency agent created with custom function tools
🔧 Available tools:
• get_fee_for_payment_method - Looks up company fee structure
• get_exchange_rate - Gets current exchange rates

⬢

↑ ↓ 🗑

```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
- await currency_runner.run_debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)

### Created new session: debug_session_id

User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
currency_agent > You will receive €455.70.

Here's how that's calculated:
1. **Fee deduction:** A fee of 2% is charged for using a platinum credit card. This means 2% of 500 USD is 10 USD.
2. **Amount after fee:** After deducting the fee, you have 500 USD - 10 USD = 490 USD remaining.
3. **Currency conversion:** With an exchange rate of 0.93, your 490 USD is converted to 490 * 0.93 = 455.70 EUR.
```

+ Code

+ Markdown

Sección 3

En esta sección se aborda un problema muy común: aunque el agente tenga instrucciones claras como “calcular el monto final después de comisiones”, los modelos de lenguaje no siempre son confiables haciendo matemáticas. Pueden equivocarse en operaciones, aplicar fórmulas inconsistentes o simplemente cometer errores numéricos.

La solución que se propone es cambiar el enfoque: en lugar de que el modelo haga los cálculos directamente, se le pide que genere código en Python que realice la operación, y luego ejecutar ese código. Esto mejora significativamente la confiabilidad, porque la ejecución de código es determinística y precisa, a diferencia del razonamiento interno del modelo.

Para lograr esto, se introduce el uso de un ejecutor de código integrado dentro de ADK. Este componente permite correr código en un entorno seguro (sandbox), aprovechando la capacidad de ejecución de código de Gemini. Así, el agente no solo “piensa”, sino que también puede delegar cálculos a un entorno más confiable.

Se crea un agente especializado llamado `calculation_agent`, cuya única función es generar código Python. Este agente tiene reglas muy estrictas: no puede dar explicaciones ni texto adicional, solo debe devolver un bloque de código que realice el cálculo y que imprima el resultado final. Además, tiene prohibido hacer el cálculo por sí mismo; su única responsabilidad es traducir el problema a código ejecutable.

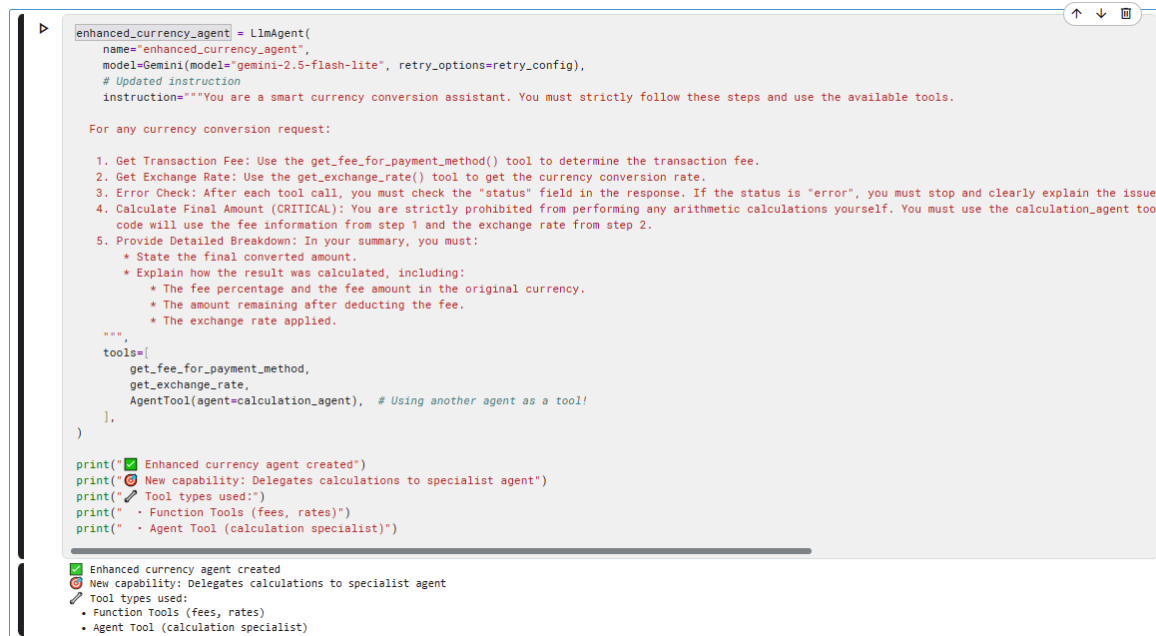

```
enhanced_currency_agent = LlmAgent(
    name="enhanced_currency_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    # Updated instruction
    instruction="""You are a smart currency conversion assistant. You must strictly follow these steps and use the available tools.

    For any currency conversion request:

    1. Get Transaction Fee: Use the get_fee_for_payment_method() tool to determine the transaction fee.
    2. Get Exchange Rate: Use the get_exchange_rate() tool to get the currency conversion rate.
    3. Error Check: After each tool call, you must check the "status" field in the response. If the status is "error", you must stop and clearly explain the issue
    4. Calculate Final Amount (CRITICAL): You are strictly prohibited from performing any arithmetic calculations yourself. You must use the calculation_agent tool
    code will use the fee information from step 1 and the exchange rate from step 2.
    5. Provide Detailed Breakdown: In your summary, you must:
        * State the final converted amount.
        * Explain how the result was calculated, including:
            * The fee percentage and the fee amount in the original currency.
            * The amount remaining after deducting the fee.
            * The exchange rate applied.

    """,
    tools=[
        get_fee_for_payment_method,
        get_exchange_rate,
        AgentTool(agent=calculation_agent), # Using another agent as a tool!
    ],
)

print("✅ Enhanced currency agent created")
print("🔗 New capability: Delegates calculations to specialist agent")
print("🔧 Tool types used:")
print("  • Function Tools (fees, rates)")
print("  • Agent Tool (calculation specialist)")
```



Después, se mejora el agente principal (por ejemplo, el de conversión de moneda). En lugar de darle una instrucción vaga como “calcula el monto final”, ahora se le indica explícitamente que genere código en Python para hacer ese cálculo y que utilice al `calculation_agent` para ejecutarlo. Esto hace el proceso más estructurado y reduce errores.

Finalmente, se integra el `calculation_agent` como una herramienta dentro del agente principal usando `AgentTool`. Esto convierte a un agente especializado en una tool reutilizable. De esta forma, el agente principal puede delegar tareas específicas —como cálculos— a otros agentes diseñados exclusivamente para eso.

La idea clave de esta sección es que la confiabilidad del sistema mejora cuando separas responsabilidades: el modelo decide qué hacer, pero el código ejecuta las operaciones críticas. Además, muestra cómo los agentes pueden trabajar juntos, donde uno actúa como herramienta de otro, creando sistemas más robustos y modulares.

Diferencia entre Agent Tools y Sub-agents

Cuando usas **Agent Tools**, un agente principal (Agent A) llama a otro agente (Agent B) como si fuera una herramienta. Agent B realiza una tarea específica y devuelve el resultado, pero no toma el control. El agente principal sigue siendo quien decide qué hacer después, integrando la respuesta dentro de su propio flujo. Es un modelo de delegación puntual: el agente pide ayuda, obtiene un resultado y continúa. Este enfoque es ideal para tareas concretas como cálculos, consultas o transformaciones de datos.

En cambio, con **Sub-Agents**, el flujo cambia completamente. Aquí, Agent A transfiere el control a Agent B, y este último se encarga de toda la interacción a partir de ese momento. El agente original deja de participar. Es más parecido a un “traspaso” que a una consulta puntual. Este patrón se utiliza cuando necesitas que un especialista maneje todo el proceso, como en sistemas de atención al cliente donde se escala a un nivel más especializado.

En el ejemplo del conversor de moneda, lo que se busca es que el agente principal siga controlando la lógica general, pero delegue los cálculos a un componente más confiable. Por eso se usan **Agent Tools**: el agente obtiene el resultado del cálculo y luego continúa con el flujo normal. Si se usaran sub-agentes, el control se perdería y no sería posible integrar fácilmente ese resultado dentro del proceso principal.

Parte 2

Model Context Protocol

En esta sección se introduce una evolución importante: hasta ahora, para que un agente interactúe con sistemas externos (como GitHub, bases de datos o Slack), era necesario programar integraciones manuales y mantener clientes de API. Eso implica más trabajo, más mantenimiento y mayor complejidad.

El Model Context Protocol (MCP) aparece como una solución a ese problema. Es un estándar abierto que permite a los agentes conectarse directamente a integraciones ya existentes, sin tener que construirlas desde cero. En lugar de escribir código específico para cada servicio, el agente simplemente se conecta a un servidor MCP que ya expone esas capacidades.

Gracias a MCP, los agentes pueden acceder a datos externos en tiempo real provenientes de APIs, bases de datos u otros servicios, pero sin necesidad de implementar integraciones personalizadas. Esto reduce significativamente el esfuerzo de desarrollo y hace que los sistemas sean más fáciles de mantener.

¿Cómo funciona?

MCP sigue un modelo cliente-servidor. Tu agente actúa como el cliente (MCP Client), mientras que los servicios externos funcionan como servidores MCP (MCP Server) que exponen distintas herramientas.

El servidor MCP es quien ofrece capacidades específicas, por ejemplo generación de imágenes, acceso a bases de datos o conexión con APIs. Cada servidor está especializado en algo y pone esas funciones a disposición mediante una interfaz estándar.

Por otro lado, el agente (cliente) no necesita saber cómo está implementada cada integración. Simplemente se conecta al servidor MCP y utiliza las herramientas disponibles como si fueran tools normales. Esto elimina la necesidad de escribir código personalizado para cada servicio externo.

Una de las claves más importantes es que todos los servidores MCP siguen la misma interfaz. Esto significa que, sin importar si el servidor accede a una base de datos, a GitHub o a cualquier otro sistema, el agente interactúa con todos de la misma manera. Esa estandarización es lo que hace que el sistema sea escalable y fácil de extender.

Usando MCP junto al agente

Primero, se elige un servidor MCP. En el ejemplo se utiliza el paquete Everything MCP Server, pensado únicamente para pruebas. Este servidor incluye varias herramientas, pero aquí se usa específicamente `getTinyImage`, que devuelve una imagen pequeña codificada en Base64. La idea no es el caso de uso en sí, sino entender cómo funciona la conexión. En un entorno real, en lugar de este servidor de prueba, usarías integraciones con servicios como mapas, mensajería o plataformas externas.

Después, se crea el `McpToolset`, que es el componente que conecta el agente con el servidor MCP. Internamente, lo que ocurre es que se ejecuta el servidor usando `npx` (el ejecutor de paquetes de Node.js) y se establece la comunicación con el agente. Aunque el servidor tenga múltiples herramientas disponibles, se puede filtrar cuáles usar, en este caso solo `getTinyImage`. Esto es importante porque permite limitar el alcance del agente y mantener el control sobre lo que puede hacer.

Una vez configurado el toolset, simplemente se añade al agente como cualquier otra herramienta. Desde ese momento, el agente puede invocar funciones del servidor MCP igual que si fueran tools locales, sin necesidad de implementar la lógica de integración.

Finalmente, se ejecuta y se prueba el agente para verificar que la conexión funciona correctamente y que puede utilizar la herramienta externa.

```
5) # MCP integration with Everything Server
   mcp_image_server = McpToolset(
       connection_params=StdioConnectionParams(
           server_params=StdioServerParameters(
               command="npx", # Run MCP server via npx
               args=[
                   "-y", # Argument for npx to auto-confirm install
                   "@modelcontextprotocol/server-everything",
               ],
               tool_filter=["getTinyImage"],
           ),
           timeout=30,
       )
   )

   print("✅ MCP Tool created")
```

✅ MCP Tool created

En este paso se integra finalmente MCP dentro del agente para que pueda usarlo en la práctica.

Lo que se hace es añadir el **MCP toolset** (es decir, la conexión al servidor MCP que ya configuraste) a la lista de **tools** del agente. Esto convierte automáticamente las capacidades del servidor —como **getTinyImage**— en herramientas disponibles para el agente, igual que cualquier otra tool local o custom.

Pero no basta con agregarlo técnicamente. También es necesario actualizar las instrucciones del agente. Aquí se le indica explícitamente que, cuando reciba solicitudes relacionadas con generar imágenes pequeñas, debe usar la herramienta correspondiente del MCP server. Esto es importante porque el modelo decide cuándo usar una tool basándose en las instrucciones y el contexto.

A partir de este punto, el agente ya puede:

- reconocer una petición relevante (por ejemplo, generar una imagen simple)
- decidir usar la tool de MCP
- ejecutar la llamada al servidor
- recibir el resultado (la imagen en Base64)
- continuar el flujo con esa información

La clave de este paso es entender que integrar MCP no solo es conectar tecnología, sino también **enseñarle al agente cuándo y cómo usarla** mediante buenas instrucciones.

```
# Create image agent with MCP integration
image_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="image_agent",
    instruction="Use the MCP Tool to generate images for user queries",
    tools=[mcp_image_server],
)
```

[+ Code](#) [+ Markdown](#)

creamos el runner

```
from google.adk.runners import InMemoryRunner

runner = InMemoryRunner(agent=image_agent)
```

Probamos el agente

```
response = await runner.run_debug("Provide a sample tiny image", verbose=True)
```

```
### Created new session: debug_session_id
```

```
User > Provide a sample tiny image
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/tools/mcp_tool/mcp_tool.py:101: UserWarning: [EXPERIMENTAL] BaseAuthenticatedTool: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
```

```
super().__init__(
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.co
ntent.parts accessor to get the full model response.
```

```
Image_agent > [Calling tool: get-tiny-image({})]
```

```
Image_agent > [Tool result: {'content': [{'type': 'text', 'text': "Here's the image you requested:"}, {'type': 'image', 'data': ...}]}
```

```
Image_agent > Here's the image you requested:
```

```
The image above is the MCP logo.
```

```
from IPython.display import display, Image as IPImage
import base64

for event in response:
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, "function_response") and part.function_response:
                for item in part.function_response.response.get("content", []):
                    if item.get("type") == "image":
                        display(IPImage(data=base64.b64decode(item["data"])))
```



Hasta ahora, el flujo típico de un agente era lineal: el usuario hace una solicitud, el agente llama a una herramienta, la herramienta responde inmediatamente y el agente devuelve el resultado. Todo ocurre en un solo ciclo sin interrupciones.

Sin embargo, en escenarios reales esto no siempre es suficiente. Hay casos donde una acción no puede ejecutarse automáticamente porque requiere tiempo, validación externa o aprobación humana. Ahí es donde entran las LRO.

La idea principal es que una herramienta puede pausar su ejecución, esperar una decisión externa y luego continuar más adelante. En lugar de terminar

inmediatamente, el flujo se detiene en un punto intermedio hasta recibir una respuesta.

Por ejemplo, en un sistema de envíos, si el usuario solicita una operación pequeña, el agente puede aprobarla automáticamente. Pero si la orden es grande, el sistema debe detenerse y pedir aprobación antes de continuar. Solo cuando un humano confirma, la operación se reanuda y se completa.

Este patrón es especialmente importante en situaciones críticas o sensibles, como transacciones financieras, operaciones masivas de datos, procesos de cumplimiento legal o acciones irreversibles como eliminar cuentas o infraestructura. En todos estos casos, es necesario un punto de control humano antes de ejecutar la acción final.

En la implementación del curso se construye un agente coordinador de envíos con una lógica simple: si el pedido es pequeño (por ejemplo, 5 contenedores o menos), se procesa automáticamente; si es grande, se detiene y solicita aprobación.

Para lograr esto, se introduce un elemento clave del framework: el `tool_context`. Este objeto es proporcionado automáticamente por ADK cuando se ejecuta la herramienta. Permite dos cosas fundamentales: por un lado, solicitar aprobación mediante `request_confirmation()`, y por otro, verificar si el usuario aprobó o rechazó la acción usando el estado de confirmación.

¿Cómo funciona?

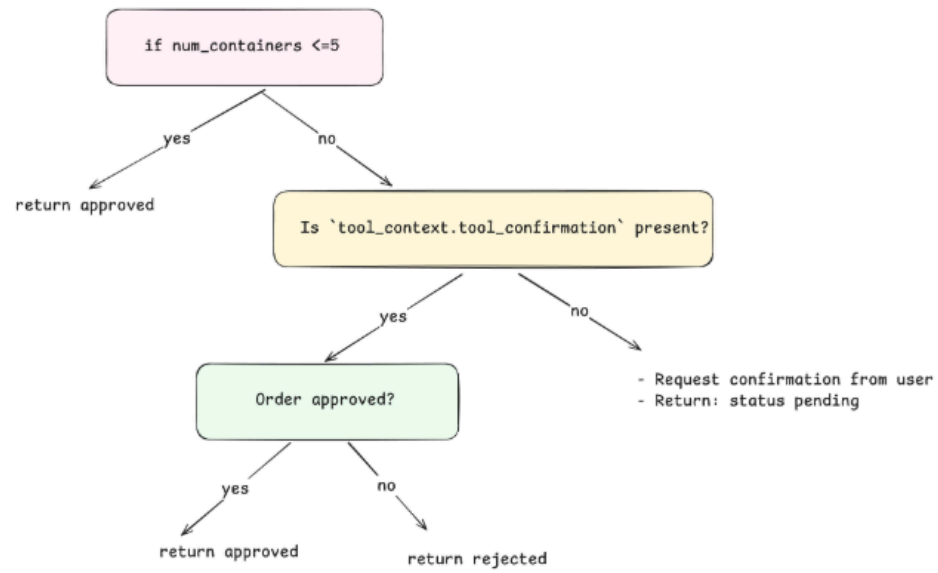
El comportamiento del tool se divide en tres escenarios, dependiendo del estado de `tool_context.tool_confirmation`, que es lo que indica si ya existe una decisión humana o no.

En el primer escenario, cuando el pedido es pequeño (por ejemplo, 5 contenedores o menos), la herramienta simplemente ejecuta la operación y devuelve el resultado inmediatamente. En este caso no se solicita ninguna aprobación y el flujo es completamente automático.

El segundo escenario ocurre cuando el pedido es grande y es la primera vez que se ejecuta la herramienta. Aquí el sistema detecta que aún no existe una confirmación (`tool_context.tool_confirmation` no está disponible). Entonces la herramienta no continúa con la operación, sino que llama a `request_confirmation()` para solicitar aprobación humana. En ese momento devuelve un estado “pendiente” y ADK genera un evento de tipo `adk_request_confirmation`, lo que provoca que la ejecución del agente se pause hasta recibir una respuesta externa.

El tercer escenario ocurre cuando el proceso se reanuda después de que el humano ya tomó una decisión. En esta segunda ejecución, el sistema detecta que ahora sí existe `tool_confirmation`. A partir de ahí revisa si la respuesta fue aprobada o rechazada. Si fue aprobada, la herramienta completa la operación; si fue rechazada, la cancela y devuelve un estado correspondiente.

La idea clave es que el mismo tool se ejecuta en dos momentos distintos: primero para solicitar aprobación y pausar el flujo, y luego para continuar usando la decisión del usuario.



`place_shipping_order` Tool

Crear el agente, la app y el runner

En esta parte se muestra cómo se arma la estructura completa para que un agente pueda trabajar con operaciones de larga duración (LRO) de forma correcta y “resumible”.

Primero se crea el agente y se le añade la tool de envío con lógica de aprobación. Lo importante aquí es que el agente no necesita manejar explícitamente la lógica de pausa o confirmación: la tool ya decide internamente cuándo pedir aprobación según el tamaño del pedido. Esto mantiene el agente simple, mientras la complejidad queda encapsulada en la herramienta.

Luego viene el punto clave: envolver el agente en una App con capacidad de resumir estado. El problema de base es que un LlmAgent normal es stateless, es decir, no recuerda nada entre llamadas. Cada ejecución es independiente, lo que hace imposible manejar procesos interrumpidos como aprobaciones humanas.

La solución es usar una App con soporte de resumabilidad, que actúa como una capa de persistencia. Esta App guarda todo el contexto necesario cuando la ejecución se pausa, incluyendo:

- todos los mensajes de la conversación hasta ese punto
- la tool que se estaba ejecutando (por ejemplo `place_shipping_order`)
- los parámetros usados (como número de contenedores o destino)
- el punto exacto donde se detuvo (esperando aprobación)

Gracias a esto, cuando el proceso se reanuda, la App reconstruye el estado completo. El agente continúa exactamente desde donde se detuvo, sin perder información ni repetir pasos, como si no hubiera pasado tiempo entre medias.

Finalmente, al crear el Session y el Runner, en lugar de pasar el agente directamente, se pasa la App (`app=shipping_app`). Esto es lo que habilita todo el sistema de resumabilidad, ya que el runner ahora sabe que debe gestionar estado persistente y no solo ejecutar llamadas independientes.

Construyendo el Workflow

En esta sección se describe cómo se construye el workflow que hace posible las operaciones de larga duración, es decir, la parte que conecta la ejecución del agente con la pausa y la reanudación controlada.

Lo primero que se aclara es algo importante: el agente por sí solo no maneja automáticamente el ciclo de pausar y continuar. Aunque la tool pueda solicitar aprobación, es el workflow externo el que tiene la responsabilidad de detectar la pausa, pedir la decisión humana y luego reanudar la ejecución.

El flujo completo siempre sigue tres pasos. Primero, el sistema debe detectar la pausa, lo cual se hace revisando los eventos generados por ADK. Si aparece un evento especial llamado `adk_request_confirmation`, significa que la ejecución se detuvo y está esperando aprobación. Segundo, el sistema debe obtener la decisión humana, que en un entorno real vendría de una interfaz (por ejemplo, un botón de aprobar/rechazar), aunque en el notebook se simula manualmente. Tercero, el sistema debe reenviar esa decisión al agente, utilizando el `invocation_id` guardado para continuar exactamente la misma ejecución.

Aquí entra el concepto de eventos. En ADK, todo lo que ocurre durante la ejecución del agente se convierte en un evento: llamadas a tools, respuestas del modelo, resultados de funciones, etc. Esto permite reconstruir paso a paso lo que ocurrió.

Dentro de estos eventos, el más importante en este flujo es `adk_request_confirmation`. Este evento es especial porque indica explícitamente que el agente ha llegado a un punto donde debe detenerse y esperar una decisión externa. Además, contiene el `invocation_id`, que es un identificador único de esa ejecución específica.

El `invocation_id` es crítico porque permite que el sistema sepa exactamente qué ejecución reanudar. Cada vez que se llama a `run_async`, se genera uno nuevo. Si no se reutiliza el mismo ID al reanudar, ADK interpretaría la acción como una ejecución completamente nueva en lugar de continuar la anterior.

Después se introducen funciones auxiliares que simplifican este flujo. Una de ellas es `check_for_approval`, que recorre todos los eventos y detecta si existe una solicitud de confirmación. Si la encuentra, extrae tanto el `approval_id` como el `invocation_id`. Si no encuentra nada, devuelve `None`.

Otra función es `print_agent_response`, que simplemente extrae y muestra el texto generado por el agente a partir de los eventos, facilitando la visualización de la salida.

Finalmente, `create_approval_response` se encarga de construir la respuesta que el sistema enviará de vuelta al agente. Toma la decisión humana (aprobar o rechazar) y la transforma en un objeto estructurado que ADK puede entender, envolviéndolo en un formato de respuesta de función.

Día 3

Gestión de sesiones

Los modelos de lenguaje utilizados en inteligencia artificial no cuentan con memoria propia, por lo que cada vez que reciben una instrucción responden únicamente con la información disponible en ese momento. Esta característica dificulta mantener conversaciones continuas o coherentes. Para resolver este problema se utilizan las sesiones, que permiten conservar el contexto de una conversación y darle continuidad.

Una sesión es un espacio donde se almacena la información de una conversación entre un usuario y un sistema. Gracias a esto, el sistema puede recordar lo que se ha dicho previamente y generar respuestas más adecuadas. Cada sesión funciona de manera independiente, por lo que no se comparte entre distintos usuarios ni entre diferentes conversaciones.

Dentro de una sesión se registran las interacciones entre el usuario y el sistema, incluyendo los mensajes enviados y las respuestas generadas. También se consideran las acciones que realiza el sistema, como el uso de herramientas externas o consultas adicionales. Además, se mantiene un estado con datos temporales que ayudan a conservar el contexto y facilitar el desarrollo de la conversación.

En aplicaciones donde existen múltiples usuarios, es necesario organizar las sesiones de manera adecuada. Para ello se implementan mecanismos que permiten crear, almacenar y recuperar la información de cada sesión. También se utiliza un componente que coordina la interacción entre el usuario y el sistema, asegurando que la conversación se mantenga ordenada y coherente.

La gestión de sesiones es fundamental porque permite que las conversaciones sean más naturales y evita que el sistema pierda el contexto. Esto mejora la experiencia del usuario y facilita el desarrollo de aplicaciones más eficientes e inteligentes.

```
APP_NAME = "default" # Application
USER_ID = "default" # User
SESSION = "default" # Session

MODEL_NAME = "gemini-2.5-flash-lite"

# Step 1: Create the LLM Agent
root_agent = Agent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot", # Description of the agent's purpose
)

# Step 2: Set up Session Management
# InMemorySessionService stores conversations in RAM (temporary)
session_service = InMemorySessionService()

# Step 3: Create the Runner
runner = Runner(agent=root_agent, app_name=APP_NAME, session_service=session_service)

print(f"✅ Stateful agent initialized!")
print(f"  - Application: {APP_NAME}")
print(f"  - User: {USER_ID}")
print(f"  - Using: {session_service.__class__.__name__}")
```

```
✅ Stateful agent initialized!
  - Application: default
  - User: default
  - Using: InMemorySessionService
```

En esta parte se pone a prueba el funcionamiento de un agente con memoria mediante el uso de una sesión. Se ejecuta una conversación con dos preguntas dentro de la misma sesión, lo que permite que el sistema conserve el contexto entre ambas interacciones.

Primero, el usuario se presenta diciendo su nombre y realiza una pregunta. El agente responde correctamente tanto al saludo como a la consulta. Después, el usuario hace una segunda pregunta relacionada con información previa, en este caso su nombre. Gracias a que ambas interacciones pertenecen a la misma sesión, el agente es capaz de recordar el dato proporcionado anteriormente y responder de forma correcta.

Esto demuestra cómo el uso de sesiones permite mantener la continuidad en una conversación, haciendo que el agente parezca tener memoria y ofreciendo respuestas más coherentes.

Sin embargo, también se identifica una limitación importante. El tipo de almacenamiento utilizado en este caso es temporal, lo que significa que toda la información se pierde una vez que la aplicación se detiene. Por ello, aunque el agente puede recordar datos durante la sesión activa, no conserva información a largo plazo.

```
# Run a conversation with two queries in the same session
# Notice: Both queries are part of the SAME session, so context is maintained
await run_session(
    runner,
    [
        "Hi, I am Sam! What is the capital of United States?",
        "Hello! What is my name?", # This time, the agent should remember!
    ],
    "stateful-agentic-session",
)
```

```
### Session: stateful-agentic-session
```

```
User > Hi, I am Sam! What is the capital of United States?
```

```
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.
```

```
User > Hello! What is my name?
```

```
gemini-2.5-flash-lite > Your name is Sam!
```


Sesiones persistentes con *DatabaseSessionService*

En esta sección se aborda el uso de sesiones persistentes, las cuales permiten que la información de una conversación no se pierda al reiniciar la aplicación. A diferencia del almacenamiento en memoria, que solo funciona mientras el sistema está activo, aquí se utiliza una base de datos para conservar los datos de forma permanente.

Se explica que existen diferentes opciones para manejar sesiones según la necesidad. Algunas son útiles solo para pruebas, mientras que otras permiten guardar la información de manera duradera o incluso escalar a sistemas más grandes. En este caso, se utiliza un servicio basado en base de datos, ya que permite mantener las conversaciones incluso después de que el sistema se reinicie.

Service	Use Case	Persistence	Best For
<code>InMemorySessionService</code>	Development & Testing	✗ Lost on restart	Quick prototypes
<code>DatabaseSessionService</code>	Self-managed apps	✓ Survives restarts	Small to medium apps
Agent Engine Sessions	Production on GCP	✓ Fully managed	Enterprise scale

Para implementar esto, se crea un agente conversacional y se configura un servicio que almacena las sesiones en una base de datos local utilizando SQLite. Esto permite guardar automáticamente toda la información de la conversación sin necesidad de configurar servidores adicionales, lo cual es ideal para proyectos pequeños o demostraciones.

```

# Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f"    - Database: my_agent_data.db")
print(f"    - Sessions will survive restarts!")

```

Al realizar una prueba, el usuario proporciona su nombre y hace una pregunta. Posteriormente, al hacer otra consulta dentro de la misma sesión, el agente es capaz de recordar el nombre correctamente. Esto demuestra que la información se está guardando y reutilizando de forma efectiva.

```

await run_session(
    runner,
    ["Hi, I am Sam! What is the capital of the United States?", "Hello! What is my name?"],
    "test-db-session-01",
)

```

```

### Session: test-db-session-01

User > Hi, I am Sam! What is the capital of the United States?
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?
gemini-2.5-flash-lite > You told me your name is Sam.

```

Después, se propone reiniciar el entorno y volver a ejecutar el sistema utilizando el mismo identificador de sesión. Al hacer esto, el agente sigue recordando la información previamente almacenada, lo que confirma que los datos persisten gracias al uso de la base de datos.

También se verifica que las sesiones son independientes entre sí. Al crear una nueva sesión con un identificador distinto, el agente no recuerda información anterior, lo que demuestra que cada conversación se mantiene aislada y privada.

Finalmente, se menciona que toda la información de las conversaciones se almacena en la base de datos en forma de eventos, los cuales incluyen tanto los mensajes del usuario como las respuestas del agente. Esto permite consultar, analizar o reutilizar los datos en cualquier momento.

Compactación de contexto

En esta sección se introduce el concepto de compactación de contexto, que surge como solución a un problema común cuando se almacenan todas las interacciones de una conversación. A medida que una sesión crece, la cantidad de eventos acumulados aumenta considerablemente, lo que puede afectar el rendimiento del sistema y generar mayores costos.

Para abordar esto, se utiliza una técnica que permite resumir automáticamente partes de la conversación. En lugar de conservar todo el historial completo, el sistema reemplaza segmentos antiguos por un resumen más corto que mantiene la información relevante. Esto permite que el agente siga entendiendo el contexto sin necesidad de procesar grandes cantidades de datos.

Para implementar esta funcionalidad, se configura una aplicación que define cada cuántas interacciones se debe realizar la compactación y cuánta información reciente se debe conservar sin resumir. A partir de esta configuración, el sistema monitorea la conversación y, cuando se alcanza el límite establecido, ejecuta el proceso de resumen de manera automática y sin intervención del usuario.

Durante la ejecución, la conversación parece normal, pero internamente el sistema ya está optimizando el contexto. Posteriormente, al revisar los datos almacenados en la sesión, se puede observar que existe un evento especial que contiene el resumen generado, lo que confirma que la compactación ocurrió correctamente.

Este proceso permite mejorar la eficiencia del sistema, ya que reduce la cantidad de información que el modelo necesita procesar en cada interacción. Además, ayuda a mantener el enfoque en los aspectos más importantes de la conversación, evitando que detalles irrelevantes afecten las respuestas.

También se menciona que este mecanismo puede personalizarse, permitiendo definir cómo se generan los resúmenes o incluso utilizar modelos distintos para

hacerlo. Otra opción disponible es el uso de almacenamiento en caché para optimizar aún más el manejo de información.

Finalmente, se plantea un nuevo reto relacionado con la gestión del contexto, ya que en algunos casos no es necesario compartir todo el historial de una sesión, sino únicamente ciertos datos importantes. Esto abre la puerta a nuevas estrategias para mejorar la experiencia del usuario mediante un manejo más inteligente de la información.

Día 4

En esta sección se trabaja el proceso de ajuste fino de un modelo de inteligencia artificial para adaptarlo a una tarea específica. A diferencia de usar un modelo general, aquí se entrena con ejemplos concretos para que aprenda a resolver un problema de forma más precisa, en este caso clasificar textos según su categoría.

Primero se configura el entorno y se obtiene acceso a la API necesaria para trabajar con el modelo. Después, se selecciona un modelo base compatible con ajuste fino, el cual servirá como punto de partida para el entrenamiento.

Posteriormente, se utiliza un conjunto de datos compuesto por publicaciones de distintos temas. Cada texto pertenece a una categoría específica, lo que permite entrenar al modelo para que aprenda a identificar patrones y clasificar correctamente nuevas entradas. Antes de usar los datos, se realiza un preprocesamiento para limpiar el contenido, eliminar información innecesaria y hacerlo más uniforme.

Una vez preparados los datos, se reduce su tamaño para facilitar el entrenamiento, seleccionando solo una parte representativa de cada categoría. Esto permite trabajar de manera más eficiente sin necesidad de grandes recursos.

Antes de entrenar el modelo, se evalúa su rendimiento inicial utilizando el modelo base. Se observa que, aunque puede dar respuestas relacionadas, no siempre clasifica correctamente los textos y tiende a generar respuestas largas o poco precisas. Esto sirve como punto de comparación para medir la mejora después del ajuste.

Luego se inicia el proceso de entrenamiento, donde el modelo aprende a partir de los ejemplos proporcionados. Durante este proceso se pueden ajustar parámetros como el número de veces que se revisan los datos, el tamaño de los grupos de

entrenamiento y la velocidad de aprendizaje. El entrenamiento se ejecuta en segundo plano y puede tardar varios minutos o más, dependiendo de la carga del sistema.

Una vez finalizado el ajuste, el modelo resultante es capaz de clasificar textos de manera directa, sin necesidad de instrucciones adicionales. Al probarlo con nuevos datos, se observa que responde de forma más precisa y concisa.

Finalmente, se evalúa nuevamente el modelo utilizando datos de prueba. Los resultados muestran una mejora significativa en la precisión en comparación con el modelo original. Además, el modelo ajustado utiliza menos recursos, ya que requiere menos texto de entrada y genera respuestas más cortas, lo que reduce costos y mejora la eficiencia.

Día 5

El video explica cómo ajustar un modelo de inteligencia artificial para que realice una tarea específica de forma más eficiente. En lugar de usar un modelo general que responde de manera amplia, se entrena con ejemplos concretos para que aprenda exactamente qué hacer, en este caso clasificar textos según su categoría.

Primero se muestra que un modelo sin ajustar puede entender el contenido, pero no siempre responde correctamente o lo hace de forma muy extensa. Esto evidencia la necesidad de especializarlo. Después, se preparan los datos de entrenamiento, limpiando el texto y organizándolo en pares de entrada y salida, donde cada texto tiene una categoría asignada.

Luego se realiza el proceso de fine-tuning, en el que el modelo aprende a partir de estos ejemplos. Este entrenamiento no modifica completamente el modelo, sino que ajusta ciertos parámetros para adaptarlo a la tarea sin requerir grandes recursos.

Una vez terminado, el modelo ajustado puede clasificar textos directamente, sin necesidad de instrucciones adicionales. Además, sus respuestas son más cortas, precisas y consistentes.

Finalmente, se comparan los resultados antes y después del ajuste, mostrando que el modelo mejorado tiene mayor precisión y utiliza menos recursos, lo que lo hace más eficiente para aplicaciones reales.

En resumen, el video demuestra que el fine-tuning es una técnica clave para convertir un modelo general en una herramienta especializada, mejorando su desempeño y reduciendo costos.